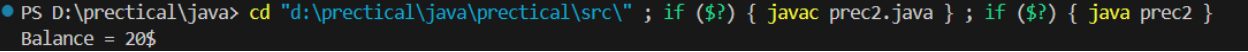


CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY**DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY & RESEARCH**

Department of Computer Science & Engineering

Subject Name: Java Programming**Semester:** 3rd**Subject Code:** CSE201**Academic year:** 2024-25**Part - 1**

No.	Data Types, Variables, String, Control Statements, Operators, Arrays
2.	Imagine you are developing a simple banking application where you need to display the current balance of a user account. For simplicity, let's say the current balance is \$20. Write a java program to store this balance in a variable and then display it to the user. <u>PROGRAM CODE:</u> <pre>public class prec2 { public static void main(String[] args) { int a = 20; System.out.println("Balance = "+a+"\$"); } }</pre> <u>OUTPUT:</u>  <u>CONCLUSION:</u> Learnt about the basics of java and basic syntax.

3. Write a program to take the user for a distance (in meters) and the time taken (as three numbers: hours, minutes, seconds), and display the speed, in meters per second, kilometers per hour and miles per hour (hint:1 mile = 1609 meters).

PROGRAM CODE:

```
import java.util.*;

public class prec3 {

    public static void main(String[] args) {
        float time[] = new float[3];
        Scanner s = new Scanner(System.in);
        int choice;
        float dis, tmin, tsec, thour;
        float calculate;

        System.out.print("Enter Distance : ");
        dis = s.nextFloat();

        System.out.print("Enter Time In Hours : ");
        time[0] = s.nextFloat();
        System.out.print("Enter Time In Min : ");
        time[1] = s.nextFloat();
        System.out.print("Enter Time In Sec : ");
        time[2] = s.nextFloat();

        tmin = (time[0] * 60);
        tsec = ((tmin + time[1]) * 60) + time[2];
        thour = time[0] + (time[1] / 60) + (time[2] / 3600);

        System.out.println("Choose Unit To Display Velocity");
        System.out.println("1.m/s");
        System.out.println("2.km/h");
        System.out.println("3.miles/h");
        choice = s.nextInt();
```

```

switch (choice) {
    case 1:

        calculate = dis / tsec;
        System.out.println("Velocity = " +
calculate + " m/s");
        break;

    case 2:
        dis = dis / 1000;
        calculate = dis / thour;
        System.out.println("Velocity = " +
calculate + " km/h");
        break;

    case 3:
        dis = dis / 1609;
        calculate = dis / thour;
        System.out.println("Velocity = " +
calculate + " miles/h");
        break;
    }
}
}

```

OUTPUT:

```

Enter Distance : 1000
Enter Time In Hours : 1
Enter Time In Min : 0
Enter Time In Sec : 0
Choose Unit To Display Velocity
1.m/s
2.km/h
3.miles/h
2
Velocity = 1.0 km/h

```

```
Enter Distance : 1000
Enter Time In Hours : 1
Enter Time In Min : 0
Enter Time In Sec : 0
Choose Unit To Display Velocity
1.m/s
2.km/h
3.miles/h
1
Velocity = 0.2777778 m/s
```

```
Enter Distance : 1235
Enter Time In Hours : 1
Enter Time In Min : 2
Enter Time In Sec : 5
Choose Unit To Display Velocity
1.m/s
2.km/h
3.miles/h
3
Velocity = 0.74180055 miles/h
```

CONCLUSION: In this practical we learnt about scanner class by using it we can take input from the user.

4. Imagine you are developing a budget tracking application. You need to calculate the total expenses for the month. Users will input their daily expenses, and the program should compute the sum of these expenses. Write a Java program to calculate the sum of elements in an array representing daily expenses.

PROGRAM CODE:

```
import java.util.Scanner;

public class prec4 {
    public static void main(String[] args) {

        int i;
        float sum=0;
        float arr[]=new float[5];

        Scanner s = new Scanner(System.in);

        System.out.println("Enter Your Expenses :
");
        for(i=0;i<5;i++)
```

```

    {
        arr[i]=s.nextFloat();
        sum+=arr[i];
    }
    System.out.println("Total Expense = " +
sum);
    }
}

```

OUTPUT:

```

Enter Your Expenses :
1000
2000
5686
562
462
Total Expense = 9710.0

```

CONCLUSION: Learnt about array, how array works in java, ways to initialize array.

5. An electric appliance shop assigns code 1 to motor,2 to fan,3 to tube and 4 for wires. All other items have code 5 or more. While selling the goods, a sales tax of 8% to motor,12% to fan,5% to tube light,7.5% to wires and 3% for all other items is charged. A list containing the product code and price in two different arrays. Write a java program using switch statement to prepare the bill.

PROGRAM CODE:

```

import java.util.Scanner;

public class prec5 {

    public static void main(String[] args) {
        float sum = 0;
        float price[] = {100, 100, 100, 100, 100};
        int choice, qun;
        Scanner s = new Scanner(System.in);

        System.out.println("1.Motor");
        System.out.println("2.Fan");
        System.out.println("3.Tube");
    }
}

```

```

        System.out.println("4.Wires");
        System.out.println("5.Other");
        System.out.println("Enter Item You Want
to Add : ");
        choice = s.nextInt();

        System.out.println("Total Quntity : ");
        qun = s.nextInt();

        switch (choice) {
            case 1:

                sum += (price[0]+(price[0] * 0.08)) *
qun;

                break;
            case 2:

                sum += (price[1]+(price[1] * 0.12)) *
qun;

                break;
            case 3:

                sum += (price[2]+(price[2] * 0.05))*
qun;

                break;

            case 4:

                sum += (price[3]+(price[3] * 0.075))
* qun;

                break;

            case 5:

                sum += (price[4]+(price[4] * 0.03)) *
qun;

```

```

        break;
    }

    System.out.println("Total = " + sum);

}
}

```

OUTPUT:

```

1.Motor
2.Fan
3.Tube
4.Wires
5.Other
Enter Item You Want to Add :
1
Total Quntity :
5
Total = 540.0

```

CONCLUSION: Learnt about the switch case.

6. Create a Java program that prompts the user to enter the number of days (n) for which they want to generate their exercise routine. The program should then calculate and display the first n terms of the Fibonacci series, representing the exercise duration for each day.

PROGRAM CODE:

```

import java.util.*;

public class prec6{
    public static void main(String[] args) {

        int i,n;
        long sum=0,ref1=0,ref2=1,temp;
        System.out.println("Enter Total No Of Fibbonaci Value");
        Scanner s = new Scanner(System.in);
        n=s.nextInt();

        System.out.println(ref1 + "\n" + ref2);
        for(i=1;i<=n;i++)

```

```

{
    temp=ref1+ref2;
    ref1=ref2;
    ref2=temp;

    System.out.println(temp);
    sum+=temp;
}

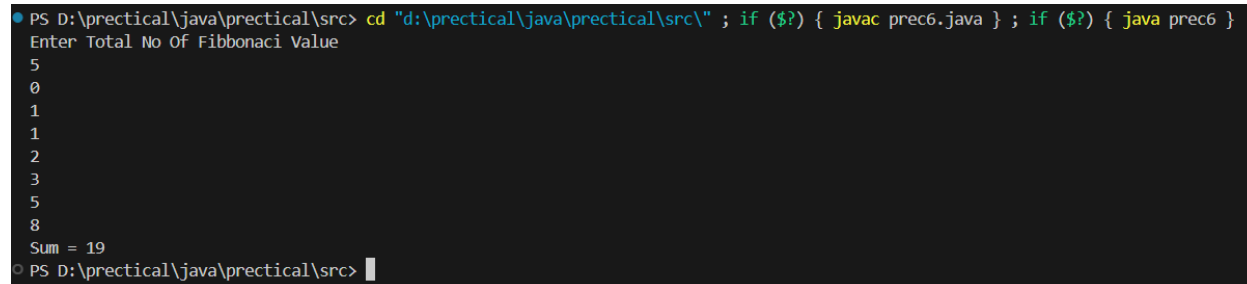
```

```

    System.out.println("Sum = " + sum );
}
}

```

OUTPUT:



```

PS D:\prectical\java\prectical\src> cd "d:\prectical\java\prectical\src\" ; if ($?) { javac prec6.java } ; if ($?) { java prec6 }
Enter Total No Of Fibbonaci Value
5
0
1
1
2
3
5
8
Sum = 19
PS D:\prectical\java\prectical\src>

```

CONCLUSION: In this practical we learnt about the implementation of Fibonacci Series.

Part - 2

No.	Strings
7	Given a string and a non-negative int n, we'll say that the front of the string is the first 3 chars, or whatever is there if the string is less than length 3. Return n copies of the front; front_times('Chocolate', 2) → 'ChoCho' front_times('Chocolate', 3) → 'ChoChoCho' front_times('Abc', 3) → 'AbcAbcAbc' <u>PROGRAM CODE:</u> <pre>import java.util.Scanner; public class prec7 { public static void main(String[] args) { String n; Scanner s = new Scanner(System.in); System.out.println("Enter String = "); n = s.nextLine(); String n1 = n.substring(0, 3); front_times(n1, 2); } static void front_times(String n, int a) { int i; for (i = 0; i < a; i++) { System.out.print(n); } } }</pre> <u>OUTPUT:</u>

```
PS D:\prectical\java\prectical\src> cd d:\prectical\java\prectical\src ; if ($?) { javac prec7.java } ; if ($?) { java prec7 }
Enter String =
Chocolate
ChoCho
```

CONCLUSION: Learnt about strings and their functions.

- 8 Given an array of integers, return the number of 9's in the array. array_count9([1, 2, 9]) → 1 array_count9([1, 9, 9]) → 2 array_count9([1, 9, 9, 3, 9]) → 3

PROGRAM CODE:

```
public class prec8 {

    public static void main(String[] args) {

        int[] arr={1,2,4,7,8,9,9,9};
        count9(arr);

    }

    static void count9(int[]arr)
    {
        int count=0;

        for(int i=0;i<arr.length;i++)
        {
            if(arr[i]==9)
            {
                count++;
            }
        }

        System.out.println("Total Count = "+
```

```
count );
```

```
}
```

```
}
```

OUTPUT:

```
PS D:\prectical\java\prectical\src> cd "d:\prectical\java\prectical\src\" ; if ($?) { javac prec8.java } ; if ($?) { java prec8 }
Total Count = 3
```

CONCLUSION: In this practical we created function which it counts total numbers of 9 in the given array.

- 9 Given a string, return a string where for every char in the original, there are two chars. double_char('The') → 'TThh ee' double_char('AAbb') → 'AAAAbbbb' double_char('Hi-There') → 'HHii--TThheerree'

PROGRAM CODE:

```
import java.util.Scanner;
```

```
public class prec9 {
```

```
    public static void main(String[] args) {
```

```
        Scanner s1 = new Scanner(System.in);
        System.out.print("Enter your string: ");
        String str = s1.nextLine();
        for (int i = 0; i < str.length(); i++) {
            char c = str.charAt(i);
            System.out.print(c + "" + c);
        }
    }
```

```
}
```

OUTPUT:

```
PS D:\prectical\java\prectical\src> cd "d:\prectical\java\prectical\src\" ; if ($?) { javac prec9.java } ; if ($?) { java prec9 }
Enter your string: The
TThhee
PS D:\prectical\java\prectical\src> █
```

CONCLUSION: In this practical we used charAt function to access each character of string and double them.

10 Perform following functionalities of the string:

- Find Length of the String
- Lowercase of the String
- Uppercase of the String
- Reverse String
- Sort The String

PROGRAM CODE:

```
import java.util.Scanner;
```

```
class prec10 {
```

```
    public static void main(String[] args) {
```

```
        String a;
```

```
        int i, j;
```

```
        char ch1;
```

```
        Scanner s = new Scanner(System.in);
```

```
        System.out.println("Enter String : ");
```

```
        a = s.nextLine();
```

```
        char[] c = a.toCharArray();
```

```
        System.out.println("String Length = " + a.length());
```

```
        System.out.println("Lower String = " + a.toLowerCase());
```

```
        System.out.println("Upper String = " + a.toUpperCase());
```

```
        System.out.println("Reversed String = ");
```

```
        for (i = a.length() - 1; i >= 0; i--) {
```

```
            System.out.print(a.charAt(i));
```

```
        }
```

```
        System.out.println("\nSorted String = ");
```

```
        for (i = 0; i < a.length()-1; i++) {
```

```
            for (j = 0; j < a.length() - i-1; j++) {
```

```
                if (c[j] > c[j + 1]) {
```

```
                    ch1 = c[j];
```

```
                    c[j] = c[j + 1];
```

```
                    c[j + 1] = ch1;
```

```
                }
```

```
            }
```

```
        }
```

```
        for(i=0;i<a.length();i++)
```

```
        {
```

```
            System.out.print(c[i]);
```

```

    }
}
}

```

OUTPUT:

```

Enter String :
Rudra Bhavsar
String Length = 13
Lower String = rudra bhavsar
Upper String = RUDRA BHAVSAR
Reversed String =
rasvahB arduR
Sorted String =
BRaaadhrrsuv
PS D:\prectical\java\prectical\src>

```

CONCLUSION: In this practical we used multiple string manipulation funtions.

11

Perform following Functionalities of the string: "CHARUSAT UNIVERSITY"

- Find length
- Replace 'H' by 'FIRST LATTER OF YOUR NAME'
- Convert all character in lowercase

PROGRAM CODE:

```

public class prec11 {
    public static void main(String[] args) {
        String a = "CHARUSAT UNIVERSITY";

        System.out.println("Length Of String = " + a.length());

        char [] c = a.toCharArray();

        for(int i=0;i<a.length();i++)
        {
            if(c[i]=='H')
            {
                c[i]='R';
            }
        }

        for(int i=0;i<a.length();i++)
        {
            System.out.print(c[i]);
        }

        System.out.println("\nLowered String = ");
    }
}

```

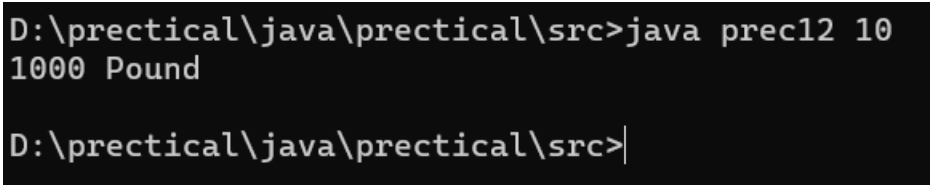
```
for (int i = 0; i < a.length(); i++) {  
    if(c[i]>='A' && c[i]<'Z')  
    {  
        c[i]+=32;  
    }  
    System.out.print(c[i]);  
}  
}}
```

OUTPUT:

```
Length Of String = 19  
CRARUSAT UNIVERSITY  
Lowered String =  
crarusat university  
PS D:\prectical\java\prectical\src> 
```

CONCLUSION: In this practical we learnt charArray function.

Part - 3

No.	Object Oriented Programming: Classes, Methods, Constructors
12.	Imagine you are developing a currency conversion tool for a travel agency. This tool should be able to convert an amount in Pounds to Rupees. For simplicity, we assume the conversion rate is fixed: 1 Pound = 100 Rupees. The tool should be able to take input both from command-line arguments and interactively from the user. <u>PROGRAM CODE:</u> <pre>public class prec12 { public static void main(String []args) { int a = Integer.parseInt(args[0]); int c = a*100; System.out.println(c + " Pound"); } }</pre> <u>OUTPUT:</u>  <pre>D:\prectical\java\prectical\src>java prec12 10 1000 Pound D:\prectical\java\prectical\src></pre> <u>CONCLUSION:</u> Learnt about the working of command line argument.

13. Create a class called Employee that includes three pieces of information as instance variables—a first name (type String), a last name (type String) and a monthly salary (double). Your class should have a constructor that initializes the three instance variables. Provide a set and a get method for each instance variable. If the monthly salary is not positive, set it to 0.0. Write a test application named EmployeeTest that demonstrates class Employee's capabilities. Create two Employee objects and display each object's yearly salary. Then give each Employee a 10% raise and display each Employee's yearly salary again.

PROGRAM CODE:

```
import java.util.Scanner;

class employee {
    String fn, ln;
    double salary;
    Scanner s = new Scanner(System.in);

    employee() {
    }

    employee(String f, String l, double sl) {
        fn = f;
        ln = l;
        salary = sl;
    }

    void setfirstname()
    {
        System.out.println("Enter Employee's First Name : ");
        fn=s.nextLine();
    }

    void setlastname()
    {
        System.out.println("Enter Employee's Last Name : ");
        ln=s.nextLine();
    }

    void setsalary()
    {
        System.out.println("Enter Employee's Salary : ");
        salary=s.nextDouble();
        if(salary<0)
```



```

        {
            salary=0;
        }
    }

    String getfirstname()
    {
        return fn;
    }
    String getlastname()
    {
        return ln;
    }
    double getsalary()
    {
        return salary;
    }
}

public class prec13 {
    public static void main(String[] args) {

        employee e1=new employee();
        employee e2=new employee();

        e1.setfirstname();
        e1.setlastname();
        e1.setsalary();

        System.out.println("First Name : " + e1.getfirstname());
        System.out.println("Last Name : " + e1.getlastname());
        System.out.println("Salary : " + e1.getsalary());
        System.out.println("New Salary : " + (e1.getsalary()+(e1.getsalary()*0.1)));

        System.out.println("-----");

        e2.setfirstname();
        e2.setlastname();
        e2.setsalary();

        System.out.println("First Name : " + e2.getfirstname());
        System.out.println("Last Name : " + e2.getlastname());
    }
}

```

```

System.out.println("Salary : " + e2.getsalary());
System.out.println("New Salary : " + (e2.getsalary()+(e2.getsalary()*0.1)));

}
}

```

OUTPUT:

```

PS D:\prectical\java\prectical\src> cd "d:\prectical\java\prectical\src\" ; if ($?) { javac prec13.java } ; if ($?) { java prec13 }
Enter Employee's First Name :
Rudra
Enter Employee's Last Name :
Bhavsar
Enter Employee's Salary :
50000
First Name : Rudra
Last Name : Bhavsar
Salary : 50000.0
New Salary : 55000.0
-----
Enter Employee's First Name :
Deep
Enter Employee's Last Name :
Talati
Enter Employee's Salary :
50000
First Name : Deep
Last Name : Talati
Salary : 50000.0
New Salary : 55000.0

```

CONCLUSION: In this practical we learnt about constructors and methods.

14. Create a class called Date that includes three pieces of information as instance variables—a month (type int), a day (type int) and a year (type int). Your class should have a constructor that initializes the three instance variables and assumes that the values provided are correct. Provide a set and a get method for each instance variable. Provide a method displayDate that displays the month, day and year separated by forward slashes (/). Write a test application named DateTest that demonstrates class Date's capabilities

PROGRAM CODE:

```

import java.util.Scanner;

class date {
    int date, month, year;
    Scanner s = new Scanner(System.in);

    date() {
    }

    date(int dt, int m, int ye) {
        date = dt;
        month = m;
        year = ye;
    }

    void setdate() {

```

```

        System.out.println("Enter Date : ");
        date = s.nextInt();
    }

    void setmonth() {
        System.out.println("Enter Month : ");
        month = s.nextInt();
    }

    void setyear() {
        System.out.println("Enter Year : ");
        year = s.nextInt();
    }

    void putdate() {
        System.out.println(date + "/" + month + "/" + year);
    }
}

public class prec14 {
    public static void main(String[] args) {

        date d1 = new date();

        d1.setdate();
        d1.setmonth();
        d1.setyear();
        d1.putdate();

    }
}

```

OUTPUT:

```

PS D:\prectical\java\prectical\src> cd "d:\prectical\java\prectical\src\" ; if ($?) { javac prec14.java } ; if ($?) { java prec14 }
Enter Date :
07
Enter Month :
09
Enter Year :
2005
7/9/2005

```

CONCLUSION: In this practical we created class by which we can see date in DD/MM/YY format.

15. Write a program to print the area of a rectangle by creating a class named 'Area' taking the values of its length and breadth as parameters of its constructor and having a method named 'returnArea' which returns the area of the rectangle. Length and breadth of rectangle are entered through keyboard.

PROGRAM CODE:

```
class area {  
  
    int length, breadth;  
  
    area() {  
    }  
  
    area(int len, int brth) {  
        length=len;  
        breadth=brth;  
    }  
  
    int getarea()  
    {  
        return length*breadth;  
    }  
}  
  
public class prec15 {  
    public static void main(String[] args) {  
  
        area a1=new area(50, 20);  
        System.out.println("Area : "+ a1.getarea());  
    }  
}
```

OUTPUT:

```
PS D:\prectical\java\prectical\src> cd "d:\prectical\java\prectical\src\" ; if ($?) { javac prec15.java } ; if ($?) { java prec15 }  
Area : 1000  
PS D:\prectical\java\prectical\src>
```

CONCLUSION: In this practical we created a class by which we can calculate area of rectangle.

16. Print the sum, difference and product of two complex numbers by creating a class named 'Complex' with separate methods for each operation whose real and imaginary parts are entered by user.

PROGRAM CODE:

```
import java.util.Scanner;

class complex
{
    int r,i;
    Scanner s=new Scanner(System.in);

    void getvalue()
    {
        System.out.println("Enter Real Part ");
        r=s.nextInt();
        System.out.println("Enter Imaginary Part ");
        i=s.nextInt();
    }

    void add(complex cx)
    {
        int sumr=r+cx.r;
        int sumi=i+cx.i;

        System.out.println("Sum :"+ sumr + "+" + sumi + "i");
    }

    void sub(complex cx)
    {
        int sumr=r-cx.r;
        int sumi=i-cx.i;

        System.out.println("Subtraction :"+ sumr + "+" + sumi + "i");
    }

    void mul(complex cx)
    {
        int sumr=(r*cx.r)-(i*cx.i);
        int sumi=(r*cx.r)+(i*cx.i);

        System.out.println("multiplication :"+ sumr + "+" + sumi + "i");
    }
}
```

```

}

public class prec16 {
    public static void main(String[] args) {

        complex c1=new complex();
        complex c2=new complex();

        c1.getvalue();
        c2.getvalue();
        c1.add(c2);
        c1.sub(c2);
        c1.mul(c2);

    }
}

```

OUTPUT:

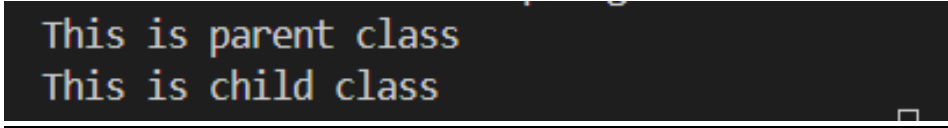
```

PS D:\prectical\java\prectical\src> cd "d:\prectical\java\Java\Set-3\" ; if ($?) { javac prec16.java } ; if ($?) { java prec16 }
Enter Real Part
2
Enter Imaginary Part
3
Enter Real Part
1
Enter Imaginary Part
2
Sum :3+5i
Subtraction :1+1i
multiplication :-4+8i

```

CONCLUSION: In this practical we created a class by which we can do addition, subtraction and multiplication of complex numbers.

PART-4

NO.	AIM OF THE PRACTICAL
17.	Create a class with a method that prints "This is parent class" and its subclass with another method that prints "This is child class". Now, create an object for each of the class and call 1 - method of parent class by object of parent <u>PROGRAM:</u> <pre>public class Main_17 { public Main_17() { } public static void main(String[] var0) { Parent var1 = new Parent(); var1.printParent(); Child var2 = new Child(); var2.printChild(); } }</pre> <u>OUTPUT:</u>  <u>CONCLUSION:</u> By creating an object of the Parent class and calling its method, we demonstrate that the method of the parent class can be called directly using the parent object. This showcases how object-oriented programming works, where a class contains methods that can be accessed through its objects.

NO.	AIM OF THE PRACTICAL
18.	Create a class named 'Member' having the following members: Data members 1 - Name 2 - Age 3 - Phone number 4 - Address 5 – Salary It also has a method named 'printSalary' which prints the salary of the members. Two classes 'Employee' and 'Manager' inherits the 'Member' class. The 'Employee' and 'Manager' classes have data members 'specialization' and 'department' respectively. Now, assign name, age, phone number, address and salary to an employee and a manager by making an object of both of these classes and print the same. <u>PROGRAM:</u> <pre>// Member class class Member { String name; int age; String phoneNumber; String address; double salary; void printSalary() { System.out.println("Salary: " + salary); } } class Employee extends Member { String specialization; void displayEmployeeDetails() { System.out.println("Employee Details:"); System.out.println("Name: " + name); System.out.println("Age: " + age); System.out.println("Phone Number: " + phoneNumber); System.out.println("Address: " + address); System.out.println("Specialization: " + specialization); printSalary(); } }</pre>


```
}

class Manager extends Member {
    String department;

    void displayManagerDetails() {
        System.out.println("Manager Details:");
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Phone Number: " + phoneNumber);
        System.out.println("Address: " + address);
        System.out.println("Department: " + department);
        printSalary();
    }
}

public class prac18 {
    public static void main(String[] args) {

        Employee employee = new Employee();
        employee.name = "JNEIL";
        employee.age = 20;
        employee.phoneNumber = "9727022105";
        employee.address = "ahemdabad";
        employee.salary = 50000.0;
        employee.specialization = "Software Engineering";
        employee.displayEmployeeDetails();

        System.out.println();

        Manager manager = new Manager();
        manager.name = "preet ";
        manager.age = 21;
        manager.phoneNumber = "8320201710";
        manager.address = "rajkot";
        manager.salary = 70000.0;
        manager.department = "blockchain expert";
        manager.displayManagerDetails();
    }
    System.out.println("23DCS013_JENIL CAPTAIN");
}
```

```
}
```

OUTPUT:

Employee Details:

Name: JENIL

Age: 20

Phone Number: 9727022105

Address: ahemdabad

Specialization: Software Engineering

Salary: 50000.0

Manager Details:

Name: preet

Age: 21

Phone Number: 8320201710

Address: rajkot

Department: blockchain expert

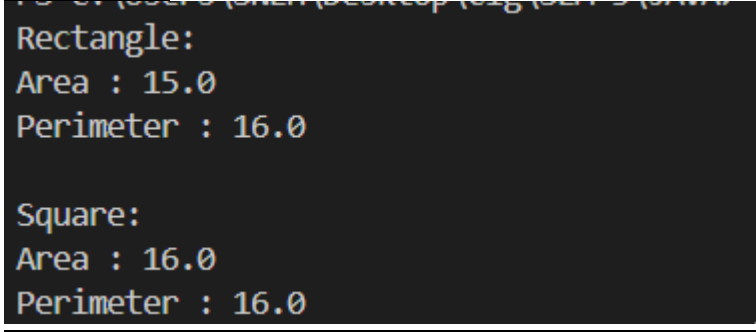
Salary: 70000.0

23DCS013_JENIL CAPTAIN

CONCLUSION:

This design allows efficient management of common attributes while providing flexibility to add specialized characteristics to different types of members.

NO.	AIM OF THE PRACTICAL
19.	Create a class named 'Rectangle' with two data members 'length' and 'breadth' and two methods to print the area and perimeter of the rectangle respectively. Its constructor having parameters for length and breadth is used to initialize length and breadth of the rectangle. Let class 'Square' inherit the 'Rectangle' class with its constructor having a parameter for its side (suppose s) calling the constructor of its parent class as 'super(s,s)'. Print the area and perimeter of a rectangle and a square. Also use array of objects. <u>PROGRAM:</u> <pre> class Rectangle { double length; double breadth; Rectangle(double length, double breadth) { this.length = length; this.breadth = breadth; } void printArea() { System.out.println("Area : " + length * breadth); } void printPerimeter() { System.out.println("Perimeter : " + 2 * (length + breadth)); } } class Square extends Rectangle { Square(double side) { super(side, side); } } public class prac19 { public static void main(String[] args) { Rectangle[] rectangles = new Rectangle[2]; rectangles[0] = new Rectangle(5.0, 3.0); System.out.println("Rectangle:"); rectangles[0].printArea(); rectangles[0].printPerimeter(); </pre>

	<pre> System.out.println(); rectangles[1] = new Square(4.0); System.out.println("Square:"); rectangles[1].printArea(); rectangles[1].printPerimeter(); } } </pre> OUTPUT:  <pre> Rectangle: Area : 15.0 Perimeter : 16.0 Square: Area : 16.0 Perimeter : 16.0 </pre> CONCLUSION: This design highlights the benefits of inheritance, code reuse, and polymorphism in object-oriented programming.
--	---

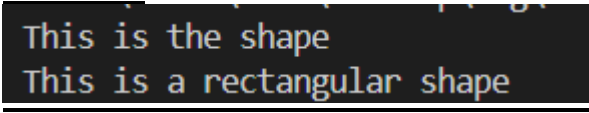
NO.	AIM OF THE PRACTICAL
20.	Create a class named 'Shape' with a method to print "This is This is a shape". Then create two other classes named 'Rectangle', and 'Circle' inheriting the Shape class, both having a method to print "This is rectangular shape" and "This is circular shape" respectively. Create a subclass 'Square' of 'Rectangle' having a method to print "Square is a rectangle". Now call the method of the 'Shape' and 'Rectangle' class by the object of 'Square' class. PROGRAM: <pre> class Shape { void display() { System.out.println("This is the shape"); } } class Rectangle extends Shape { </pre>

```
void displayRectangle()
{
    System.out.println("This is a rectangular shape");
}

class Circle extends Shape
{
    void displayCircle()
    {
        System.out.println("This is a circular shape");
    }
}

class Square extends Rectangle
{
    void displaySquare()
    {
        System.out.println("Square is a rectangle");
    }
}

public class prac20
{
    public static void main(String[] args)
    {
        Square s = new Square();
        s.display();
        s.displayRectangle();
    }
}
```

OUTPUT:

```
This is the shape
This is a rectangular shape
```

CONCLUSION:

The example provided illustrates core object-oriented programming principles, including inheritance, polymorphism, method overriding, and encapsulation. By structuring classes in a hierarchical manner that reflects logical relationships between shapes, the design promotes code reusability, flexibility, and maintainability. The use of polymorphism and method resolution order ensures that the correct behavior is invoked for each specific object type, showcasing the power and versatility of object-oriented design.

NO.	AIM OF THE PRACTICAL
21.	Create a class 'Degree' having a method 'getDegree' that prints "I got a degree". It has two subclasses namely 'Undergraduate' and 'Postgraduate' each having a method with the same name that prints "I am an Undergraduate" and "I am a Postgraduate" respectively. Call the method by creating an object of each of the three classes. <u>PROGRAM:</u> <pre>class Degree { void getDegree() { System.out.println("I got a degree!!"); } } class Postgraduate extends Degree { void getDegree() { System.out.println("I am a postgraduate!!"); } } class Undergraduate extends Degree { void getDegree() { System.out.println("I am an undergraduate!!"); } } public class prac21 { public static void main(String[] args) { Degree d = new Degree(); Postgraduate p = new Postgraduate(); Undergraduate u = new Undergraduate(); d.getDegree(); p.getDegree(); u.getDegree(); }</pre>

	<pre>}</pre> OUTPUT: <pre>I got a degree!! I am a postgraduate!! I am an undergraduate!!</pre> <u>CONCLUSION:</u> This demonstrates polymorphism in Python, where the method in the subclass overrides the method from the parent class when called on an instance of the subclass.

NO.	AIM OF THE PRACTICAL
22.	Write a java that implements an interface AdvancedArithmetic which contains amethod signature int divisor_sum(int n). You need to write a class calledMyCalculator which implements the interface. divisorSum function just takes an integer as input and return the sum of all its divisors. For example, divisors of 6 are 1, 2, 3 and 6, so divisor_sum should return 12. The value of n will be at most 1000. <u>PROGRAM:</u> <pre>import java.util.*; interface AdvancedArithmetic { int divisor_sum(int n); } class calledMyCalculator implements AdvancedArithmetic { public int divisor_sum(int n){ int sum=0; int sqrt =(int) Math.sqrt(n); for(int i=1;i<=sqrt;i++){ if(n%i==0){ sum+=i; if(i!=n/i){ sum +=n/i; } } } return sum; } } class P22 { static Scanner sc=new Scanner(System.in); public static void main(String args[]){ calledMyCalculator cmc=new calledMyCalculator(); System.out.println("enter the Number:"); int x=sc.nextInt(); System.out.println("sum:"+cmc.divisor_sum(x)); } }</pre>

OUTPUT:

```
enter the Number:  
20  
sum:42
```

CONCLUSION:

This design effectively demonstrates the implementation of interfaces in Java, as well as an efficient approach to divisor calculation.

NO.	AIM OF THE PRACTICAL
-----	----------------------

23.	Assume you want to capture shapes, which can be either circles (with a radius and a color) or rectangles (with a length, width, and color). You also want to be able to create signs (to post in the campus center, for example), each of which has a shape (for the background of the sign) and the text (a String) to put on the sign. Create classes and interfaces for circles, rectangles, shapes, and signs. Write a program that illustrates the significance of interface default method. <u>PROGRAM:</u> <pre>interface Shape { String getColor(); double getArea(); default void displayInfo() { System.out.println("Color: " + getColor()); System.out.println("Area: " + getArea()); } } class Circle implements Shape { private double radius; private String color; public Circle(double radius, String color) { this.radius = radius; this.color = color; } @Override public String getColor() { return color; } @Override public double getArea() { return Math.PI * radius * radius; } } class Rectangle implements Shape { private double length; private double width; private String color; public Rectangle(double length, double width, String color) { this.length = length;</pre>
-----	--

```
        this.width = width;
        this.color = color;
    }

    @Override
    public String getColor() {
        return color;
    }

    @Override
    public double getArea() {
        return length * width;
    }
}

class Sign {
    private Shape shape;
    private String text;

    public Sign(Shape shape, String text) {
        this.shape = shape;
        this.text = text;
    }

    public void displaySignInfo() {
        System.out.println("Sign Text: " + text);
        shape.displayInfo();
    }
}

public class prac23 {
    public static void main(String[] args) {

        Shape circle = new Circle(5, "Red");

        Shape rectangle = new Rectangle(4, 7, "Blue");

        Sign sign1 = new Sign(circle, "Welcome to Campus");
        Sign sign2 = new Sign(rectangle, "Library Ahead");

        System.out.println("Sign 1 Information:");
        sign1.displaySignInfo();

        System.out.println("\nSign 2 Information:");
        sign2.displaySignInfo();
    }
}
```

OUTPUT:

```
Sign 1 Information:  
Sign Text: Welcome to Campus  
Color: Red  
Area: 78.53981633974483
```

```
Sign 2 Information:  
Sign Text: Library Ahead  
Color: Blue  
Area: 28.0
```

CONCLUSION:**PART-V Exception Handling**

No .	Aim of the Practical
24.	AIM : Write a java program which takes two integers x & y as input, you have to compute x/y. If x and y are not integers or if y is zero, exception will occur and you have to report it. <u>PROGRAM CODE :</u> <pre>import java.util.Scanner; public class Prac_24 { public static void main(String[] args) { Scanner scanner = new Scanner(System.in); try { System.out.print("Enter integer x: "); int x = Integer.parseInt(scanner.nextLine()); System.out.print("Enter integer y: "); int y = Integer.parseInt(scanner.nextLine()); int d = x / y; System.out.println("Division " + d); } catch (NumberFormatException e) { System.out.println("Invalid input: Please enter valid integers."); } catch (ArithmeticException e) { System.out.println("Arithmetic error: Division by zero is not allowed."); } finally { scanner.close(); System.out.print("\nName : Sneh \nID : 23DCS093 "); } } }</pre> <u>OUTPUT:</u>

```
PS C:\Users\SNEH\Desktop\c1g\SEM 3\JAVA> cd C:\Users\SNEH\Desktop\c1g\SEM 3\JAVA>
Enter integer x: 12
Enter integer y: 0
Arithmetic error: Division by zero is not allowed.

Name : Sneh
ID : 23DCS093
PS C:\Users\SNEH\Desktop\c1g\SEM 3\JAVA>
```

OUTPUT: PRACTICAL-24

CONCLUSION:

This code is a simple console application that prompts the user to input two integers and performs division. It includes error handling for invalid input and division by zero. The Scanner object is used for input and is properly closed in the finally block. The program also prints a name and ID at the end. This ensures the program handles common input errors gracefully while demonstrating basic exception handling in Java.

25. AIM : Write a Java program that throws an exception and catch it using a try-catch block.

PROGRAM CODE :

```
public class Prac_25 {
    public static void main(String[] args) {
        try {
            throw new Exception("This is a custom exception");
        } catch (Exception e) {
            System.out.println("Caught an exception: " + e.getMessage());
        }
        System.out.print("\nName : Sneh \nID : 23DCS093 ");
    }
}
```

OUTPUT:

```
PS C:\Users\SNEH\Desktop\clg\SEM 3\JAVA> java pra
Caught an exception: This is a custom exception

Name : Sneh
ID : 23DCS093
PS C:\Users\SNEH\Desktop\clg\SEM 3\JAVA>
```

OUTPUT: PRACTICAL-25

CONCLUSION:

This java code demonstrates basic exception handling. It intentionally throws a custom exception and catches it, printing the exception message. The program then prints a name and ID. This example illustrates how to use try-catch blocks to manage exceptions in Java. It ensures that even when an exception occurs, the program continues to execute subsequent statements.

26. AIM : Write a java program to generate user defined exception using “throw” and “throws” keyword. Also Write a java that differentiates checked and unchecked exceptions. (Mention at least two checked and two unchecked exceptions in program).

PROGRAM CODE :

```
import java.io.IOException;
import java.sql.SQLException;
```

```
class CustomCheckedException extends Exception {
    public CustomCheckedException(String message) {
        super(message);
    }
}
```

```
class CustomUncheckedException extends RuntimeException {
    public CustomUncheckedException(String message) {
        super(message);
    }
}
```

```
public class Prac_26 {
```

```
    public static void methodThrowingCheckedException() throws
        CustomCheckedException {
        throw new CustomCheckedException("This is a custom checked exception");
    }
}
```

```
public static void methodThrowingUncheckedException() {
    throw new CustomUncheckedException("This is a custom unchecked exception");
}

public static void methodThrowingStandardCheckedExceptions() throws IOException,
SQLException {
    throw new IOException("This is an IOException");
}

public static void methodThrowingStandardUncheckedExceptions() {
    throw new NullPointerException("This is a NullPointerException");
}

public static void main(String[] args) {
    try {
        methodThrowingCheckedException();
    } catch (CustomCheckedException e) {
        System.out.println("Caught checked exception: " + e.getMessage());
    }

    try {
        methodThrowingStandardCheckedExceptions();
    } catch (IOException | SQLException e) {
        System.out.println("Caught standard checked exception: " + e.getMessage());
    }

    try {
        methodThrowingUncheckedException();
    } catch (CustomUncheckedException e) {
        System.out.println("Caught unchecked exception: " + e.getMessage());
    }

    try {
        methodThrowingStandardUncheckedExceptions();
    } catch (NullPointerException | ArithmeticException e) {
        System.out.println("Caught standard unchecked exception: " + e.getMessage());
    }
    System.out.print("\nName : Sneha \nID : 23DCS093 ");
}
}
```


OUTPUT:

```
Caught checked exception: This is a custom checked exception  
Caught standard checked exception: This is an IOException  
Caught unchecked exception: This is a custom unchecked exception  
Caught standard unchecked exception: This is a NullPointerException
```

```
Name : Sneh
```

```
ID : 23DCS093
```

```
PS C:\Users\SNEH\Desktop\c1g\SEM 3\JAVA> 
```

CONCLUSION:

This Java code demonstrates handling both checked and unchecked exceptions. It uses multiple try-catch blocks to catch custom and standard exceptions, printing appropriate messages. This ensures robust error handling and program continuity. The program concludes by printing the author's name and ID. This example effectively illustrates exception management in Java.

PART-VI File Handling & Streams

27

Write a program that will count the number of lines in each file that is specified on the command line. Assume that the files are text files. Note that multiple files can be specified, as in "java Line Counts file1.txt file2.txt file3.txt". Write each file name, along with the number of lines in that file, to standard output. If an error occurs while trying to read from one of the files, you should print an error message for that file, but you should still process all the remaining files.

PROGRAM CODE:

```
import java.io.*;

public class P27 {

    public static void main(String[] args) throws Exception {

        if (args.length == 0) {

            System.out.println("No file Found!");

        } else {

            for (int i = 0; i < args.length; i++) {

                try {

                    BufferedReader f = new BufferedReader(new FileReader(args[i]));

                    String j;

                    int count = 0;

                    while ((j = f.readLine()) != null) {

                        count++;

                    }

                    System.out.println("File name is : " + args[i] + " and Number of lines are : " + count);

                } catch (Exception e) {

                    System.out.println(e);

                }

            }

        }

    }

}
```

```
} } }
```

```
System.out.println("ID :23DCS093_SNEH PATEL");
```

```
} }
```

OUTPUT:

```
File name is : file1.txt and Number of lines are : 5
File name is : file2.txt and Number of lines are : 4
File name is : file3.txt and Number of lines are : 9
```

CONCLUSION:

This Java program reads several files named by the command line arguments and counts the number of lines in each. If no files are provided as command-line arguments, it will print out the appropriate message. Exception handling ensures graceful error management during file reading, thus a stable program.

28

Write an example that counts the number of times a particular character, such as e, appears in a file. The character can be specified at the command line. You can use xanadu.txt as the input file.

PROGRAM CODE:

```
import java.io.BufferedReader;

import java.io.FileReader;

import java.io.IOException;

public class P28{

    public static void main(String[] args) {

        if (args.length < 2) {

            System.out.println("Usage: java P28 <character> <filename>");

            return; }

    }
```

```
char targetChar = args[0].charAt(0);

String fileName = args[1];

int count = 0;

try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {

    int ch;

    while ((ch = reader.read()) != -1) {

        if (ch == targetChar) {

            count++;

        }

    }

    System.out.println("The character '" + targetChar + "' appears " + count + " times in " +
        fileName);

} catch (IOException e) {

    System.out.println("Error reading " + fileName + ": " + e.getMessage());

}

System.out.println("ID :23DCS093_SNEH PATEL");

}}
```

OUTPUT:

```
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> javac P28.java
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> java P28 a file1.txt
The character 'a' appears 16 times in file1.txt
```

CONCLUSION:

The Java program successfully counts the occurrences of a specified character in a given file, providing the result in a clear format. It handles file read errors gracefully, ensuring robust

performance even if issues arise during file access.

29

Write a Java Program to Search for a given word in a File. Also show use of Wrapper Class with an example.

PROGRAM CODE:

```
import java.io.BufferedReader;

import java.io.FileReader;

import java.io.IOException;

public class P29 {

    public static void main(String[] args) {

        if (args.length < 2) {

            System.out.println("Usage: java P29 <word> <filename>");

            return;

        }

        String searchWord = args[0];

        String fileName = args[1];

        Integer count = 0;

        try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {

            String line;

            while ((line = reader.readLine()) != null) {

                String[] words = line.split("\\W+");

                for (String word : words) {
```

```
if (word.equalsIgnoreCase(searchWord)) {  
    count++;  
} } }  
  
System.out.println("The word '" + searchWord + "' appears " + count + " times in " + fileName);  
} catch (IOException e) {  
  
    System.out.println("Error reading " + fileName + ": " + e.getMessage());  
}  
  
System.out.println("ID :23DCS093_SNEH PATEL");  
} }
```

OUTPUT:

```
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> javac P29.java  
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> java P29 and file3.txt  
The word 'and' appears 4 times in file3.txt
```

CONCLUSION:

This Java program effectively searches for a specified word in a given file and counts its occurrences. It demonstrates the use of the Integer wrapper class to manage the count, showcasing how wrapper classes can be used for object manipulation in Java.

30

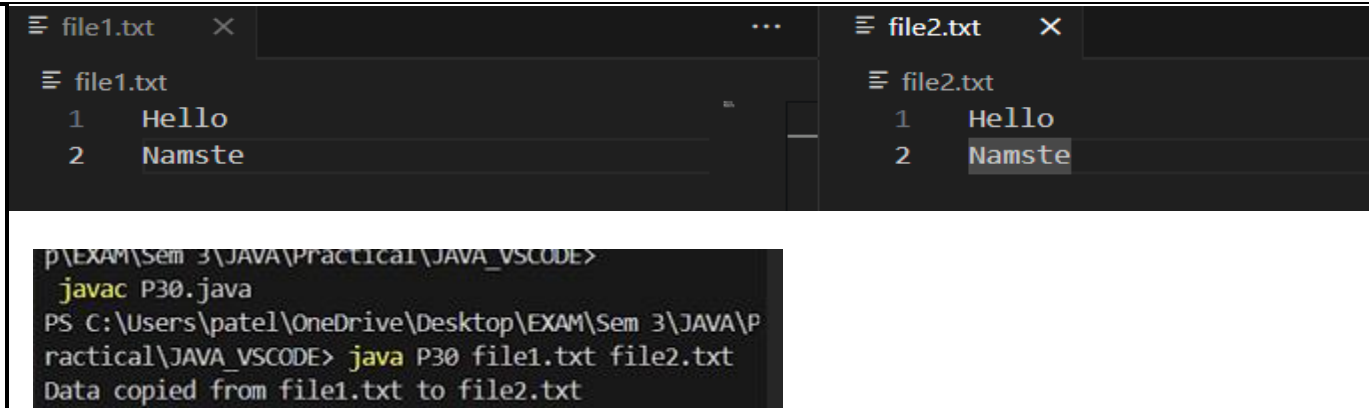
Write a program to copy data from one file to another file. If the destination file does not exist, it is created automatically.

PROGRAM CODE:

```
import java.io.FileReader;  
  
import java.io.FileWriter;  
  
import java.io.IOException;  
  
public class P30 {
```

```
public static void main(String[] args) {  
    if (args.length < 2) {  
        System.out.println("Usage: java P30 <source file> <destination file>");  
        return;  
    }  
  
    String sourceFile = args[0];  
    String destinationFile = args[1];  
  
    try (FileReader fr = new FileReader(sourceFile);  
        FileWriter fw = new FileWriter(destinationFile)) {  
        int ch;  
        while ((ch = fr.read()) != -1) {  
            fw.write(ch);  
        }  
  
        System.out.println("Data copied from " + sourceFile + " to " + destinationFile);  
    } catch (IOException e) {  
        System.out.println("Error: " + e.getMessage());  
    }  
  
    System.out.println("ID :23DCS093_SNEH PATEL");  
} }
```

OUTPUT:



```

p\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE>
javac P30.java
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> java P30 file1.txt file2.txt
Data copied from file1.txt to file2.txt

```

CONCLUSION:

This Java program efficiently copies data from a source file to a destination file, automatically creating the destination file if it does not already exist. It handles any potential I/O exceptions during the process, ensuring robust performance.

31

Write a program to show use of character and byte stream. Also show use of BufferedReader / BufferedWriter to read console input and write them into a file.

PROGRAM CODE:

```

import java.io.*;

public class P31 {

    public static void main(String[] args) {

        BufferedReader consoleReader = new BufferedReader(new InputStreamReader (System.in));

        String fileName = "output.txt";

        try (BufferedWriter fileWriter = new BufferedWriter(new FileWriter(fileName))) {

            System.out.println("Enter text (type 'exit' to finish):");

            String input;

            while (!(input = consoleReader.readLine()).equalsIgnoreCase("exit")) {

```



```
fileWriter.write(input);

fileWriter.newLine();

}

System.out.println("Data written to " + fileName);

} catch (IOException e) {

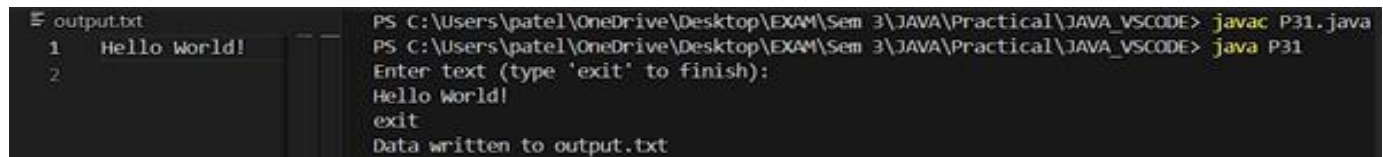
System.out.println("Error: " + e.getMessage());

}

System.out.println("ID :23DCS093_SNEH PATEL");

} }
```

OUTPUT:



```
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> javac P31.java
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> java P31
Enter text (type 'exit' to finish):
Hello World!
exit
Data written to output.txt
```

CONCLUSION:

This program effectively demonstrates the use of character streams via `BufferedReader` and `BufferedWriter` for reading console input and writing it to a file. It showcases how to handle text data efficiently while managing resources properly with `try-with-resources`.

PART-VII Multithreading

32

Write a program to create thread which display “Hello World” message. A. by extending Thread class B. by using Runnable interface.

PROGRAM CODE:

```
class Thread1 extends Thread{ // by extending Thread class

public void run(){

System.out.println("Hello world");

}}

class Thread2 implements Runnable{ //by using Runnable interface.

public void run(){

System.out.println("Hello world 1");

} }

public class P32 {

public static void main(String[] args) {

Thread1 thread = new Thread1();

thread.start();

Thread2 obj2 = new Thread2();

Thread t1 = new Thread(obj2);

t1.start();

} }
```

OUTPUT:

```
Hello world
Hello world 1
```

CONCLUSION:

This program demonstrates two approaches to creating threads in Java: extending the Thread class and implementing the Runnable interface. Both methods effectively print "Hello World," showcasing the flexibility of Java's concurrency model.

33

Write a program which takes N and number of threads as an argument. Program should distribute the task of summation of N numbers amongst number of threads and final result to be displayed on the console.

PROGRAM CODE:

```
import java.util.Scanner;

class SumTask implements Runnable {

    private int start;

    private int end;

    private static int totalSum = 0;

    public SumTask(int start, int end) {

        this.start = start;

        this.end = end;

    }

    public void run() {

        int partialSum = 0;

        for (int i = start; i <= end; i++) {

            partialSum += i;

        }

    }

}
```

```
synchronized (SumTask.class) {  
    totalSum += partialSum;  
} }  
  
public static int getTotalSum() {  
    return totalSum;  
} }  
  
public class P33 {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        System.out.print("Enter N: ");  
        int N = scanner.nextInt();  
        System.out.print("Enter number of threads: ");  
        int numThreads = scanner.nextInt();  
        Thread[] threads = new Thread[numThreads];  
        int range = N / numThreads;  
        int remainder = N % numThreads;  
        int start = 1;  
        for (int i = 0; i < numThreads; i++) {  
            int end = start + range - 1;  
            if (i == numThreads - 1) {  
                end += remainder;  
            }  
        }  
    }  
}
```

```
threads[i] = new Thread(new SumTask(start, end));  
threads[i].start();  
start = end + 1;  
}  
for (Thread thread : threads) {  
try {  
thread.join();  
} catch (InterruptedException e) {  
e.printStackTrace();  
} }  
System.out.println("Total Sum: " + SumTask.getTotalSum());  
} }
```

OUTPUT:

```
Enter N: 100  
Enter number of threads: 5  
Total Sum: 5050
```

CONCLUSION:

This program effectively demonstrates how to utilize multiple threads in Java to perform a summation task concurrently. By distributing the workload among threads, it showcases improved efficiency in computation, making it a practical example of multithreading in action.

34

Write a java program that implements a multi-thread application that has three threads. First thread generates random integer every 1 second and if the value is even, second thread computes the square of the number and prints. If the value is odd, the third thread will print the value of cube of the number.

PROGRAM CODE:

```
import java.util.Random;

class RandomNumberGenerator extends Thread {

    private final Object lock;

    public RandomNumberGenerator(Object lock) {
        this.lock = lock;
    }

    public void run() {
        Random random = new Random();

        while (true) {
            int number = random.nextInt(100);

            synchronized (lock) {
                P34.lastNumber = number;
                lock.notifyAll();
                System.out.println("Generated: " + number);
            }
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    class EvenNumberProcessor extends Thread {

        private final Object lock;

        public EvenNumberProcessor(Object lock) {
```

```
this.lock = lock;

}

public void run() {
    while (true) {
        synchronized (lock) {
            try {
                lock.wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            if (P34.lastNumber % 2 == 0) {
                int square = P34.lastNumber * P34.lastNumber;
                System.out.println("Square: " + square);
            } } } } }

class OddNumberProcessor extends Thread {
    private final Object lock;

    public OddNumberProcessor(Object lock) {
        this.lock = lock;
    }

    public void run() {
        while (true) {
            synchronized (lock) {
```

```
try {  
    lock.wait();  
} catch (InterruptedException e) {  
    e.printStackTrace();  
}  
  
if (P34.lastNumber % 2 != 0) {  
    int cube = P34.lastNumber * P34.lastNumber * P34.lastNumber;  
    System.out.println("Cube: " + cube);  
} } } } }  
  
public class P34 {  
    public static int lastNumber;  
  
    public static void main(String[] args) {  
        Object lock = new Object();  
        RandomNumberGenerator generator = new RandomNumberGenerator(lock);  
        EvenNumberProcessor evenProcessor = new EvenNumberProcessor(lock);  
        OddNumberProcessor oddProcessor = new OddNumberProcessor(lock);  
        generator.start();  
        evenProcessor.start();  
        oddProcessor.start();  
    }  
}
```

OUTPUT:


```
Generated: 43
Cube: 79507
Generated: 85
Cube: 614125
Generated: 8
Square: 64
Generated: 93
Cube: 804357
Generated: 11
Cube: 1331
Generated: 63
Cube: 250047
Generated: 80
Square: 6400
```

CONCLUSION:

This program effectively demonstrates a multi-threaded application where one thread generates random integers, while two other threads process these integers based on their parity. It highlights the use of synchronization in Java to safely share data among threads, showcasing how concurrency can be leveraged for efficient task distribution.

35

Write a program to increment the value of one variable by one and display it after one second using thread using sleep() method.

PROGRAM CODE:

```
public class P35 extends Thread {
    private int value = 0;
    public void run() {
        while (true) {
            value++;
            System.out.println("Value: " + value);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
```

```
e.printStackTrace();  
} } }  
  
public static void main(String[] args) {  
    P35 incrementer = new P35();  
    incrementer.start();  
} }
```

OUTPUT:

```
Value: 1  
Value: 2  
Value: 3  
Value: 4  
Value: 5  
Value: 6  
Value: 7  
Value: 8  
Value: 9  
Value: 10
```

CONCLUSION:

This program effectively demonstrates the use of a thread to increment a variable every second. It utilizes the sleep() method to create a delay between increments, showcasing basic thread functionality in Java.

36

Write a program to create three threads 'FIRST', 'SECOND', 'THIRD'. Set the priority of the 'FIRST' thread to 3, the 'SECOND' thread to 5(default) and the 'THIRD' thread to 7.

PROGRAM CODE:

```
class MyThread extends Thread {  
    MyThread(String name) {  
        super(name);  
    }  
}
```

```
public void run() {  
    System.out.println(getName() + " is running with priority " + getPriority());  
} }  
  
public class P36 {  
    public static void main(String[] args) {  
        MyThread first = new MyThread("FIRST");  
        MyThread second = new MyThread("SECOND");  
        MyThread third = new MyThread("THIRD");  
        first.setPriority(3);  
        first.start();  
        second.setPriority(Thread.NORM_PRIORITY);  
        second.start();  
        third.setPriority(7);  
        third.start();  
    } }  
}
```

OUTPUT:

```
FIRST is running with priority 3  
THIRD is running with priority 7  
SECOND is running with priority 5
```

CONCLUSION:

This program demonstrates thread creation and priority setting in Java by extending the Thread class. Each thread prints its name and priority when executed. Different priority levels (3, 5, 7) are set using setPriority(), showcasing the influence of priority on execution order. However, actual execution may vary due to the system's thread scheduling.

37

Write a program to solve producer-consumer problem using thread synchronization.

PROGRAM CODE:

```
class Buffer {  
    private int data;  
    private boolean isEmpty = true;  
    public synchronized void produce(int value) {  
        while (!isEmpty) {  
            try {  
                wait();  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            } }  
        data = value;  
        isEmpty = false;  
        System.out.println("Produced: " + data);  
        notify();  
    }  
    public synchronized void consume() {  
        while (isEmpty) {  
            try {  
                wait();  
            } catch (InterruptedException e) {
```

```
e.printStackTrace();
} }

System.out.println("Consumed: " + data);
isEmpty = true;
notify();
} }

class Producer extends Thread {
    private Buffer buffer;

    public Producer(Buffer buffer) {
        this.buffer = buffer;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            buffer.produce(i); // Produce values from 1 to 5
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            } } } }

class Consumer extends Thread {
    private Buffer buffer;

    public Consumer(Buffer buffer) {
```

```
this.buffer = buffer;
}

public void run() {
    for (int i = 1; i <= 5; i++) {
        buffer.consume();
        try {
            Thread.sleep(1500);
        } catch (InterruptedException e) {
            e.printStackTrace();
        } } }

public class P37 {
    public static void main(String[] args) {
        Buffer buffer = new Buffer();
        Producer producer = new Producer(buffer);
        Consumer consumer = new Consumer(buffer);
        producer.start();
        consumer.start();
    } }
```

OUTPUT:

```

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5

```

CONCLUSION:

This program demonstrates producer-consumer synchronization in Java using the wait() and notify() methods. The producer thread generates data, while the consumer thread consumes it, both synchronized to avoid race conditions. The use of wait() and notify() ensures proper coordination between the threads, allowing for controlled data production and consumption.

PART-VIII Collection Framework and Generic

38

Design a Custom Stack using ArrayList class, which implements following functionalities of stack.

My Stack -list ArrayList<Object>: A list to store elements.

isEmpty: boolean: Returns true if this stack is empty.

getSize(): int: Returns number of elements in this stack.

peek(): Object: Returns top element in this stack without removing it.

pop(): Object: Returns and Removes the top elements in this stack.

push(o: object): Adds new element to the top of this stack.

PROGRAM CODE:

```
import java.util.ArrayList;

class MyStack {

private ArrayList<Object> list = new ArrayList<>();

public boolean isEmpty() {

return list.isEmpty();

}

public int getSize() {

return list.size();

}

public Object peek() {

if (isEmpty()) {

return "Stack is empty";

}

return list.get(list.size() - 1);

}

public Object pop() {

if (isEmpty()) {

return "Stack is empty";

}

return list.remove(list.size() - 1);

}
```



```
public void push(Object o) {  
    list.add(o);  
} }  
  
public class P38 {  
    public static void main(String[] args) {  
        MyStack stack = new MyStack();  
        stack.push(10);  
        stack.push(20);  
        stack.push(30);  
        System.out.println("Top element is: " + stack.peek());  
        System.out.println("Popped element: " + stack.pop());  
        System.out.println("Popped element: " + stack.pop());  
        System.out.println("Is stack empty ? " + stack.isEmpty());  
        System.out.println("Current stack size: " + stack.getSize());  
        System.out.println("Top element now: " + stack.peek());  
    } }  
}
```

OUTPUT:

```
Top element is: 30  
Popped element: 30  
Popped element: 20  
Is stack empty ? false  
Current stack size: 1  
Top element now: 10
```

CONCLUSION:

This program demonstrates the implementation of a custom stack using the ArrayList class in Java. It provides functionalities to push, pop, peek, check if the stack is empty, and get the current size of the stack. The program effectively showcases how to manage a dynamic collection of elements while adhering to stack principles.

39

Imagine you are developing an e-commerce application. The platform needs to sort lists of products based on different criteria, such as price, rating, or name. Each product object implements the Comparable interface to define the natural ordering. To ensure flexibility and reusability, you need a generic method that can sort any array of Comparable objects. Create a generic method in Java that sorts an array of Comparable objects. This method should be versatile enough to sort arrays of different types of objects (such as products, customers, or orders) as long as they implement the Comparable interface.

PROGRAM CODE:

```
import java.util.Arrays;

public class P39 {

    public static <T extends Comparable<T>> void sortArray(T[] array) {
        Arrays.sort(array);
    }

    public static void main(String[] args) {
        Integer[] numbers = {5, 3, 9, 1, 7};
        System.out.println("Before sorting (Integers): " + Arrays.toString(numbers));
        sortArray(numbers);
        System.out.println("After sorting (Integers): " + Arrays.toString(numbers));
        String[] names = {"John", "Alice", "Bob", "David"};
        System.out.println("\nBefore sorting (Strings): " + Arrays.toString(names));
        sortArray(names);
    }
}
```

```
System.out.println("After sorting (Strings): " + Arrays.toString(names));

Product[] products = {
    new Product("Laptop", 1000),
    new Product("Phone", 800),
    new Product("Tablet", 600),
    new Product("Smartwatch", 200)
};

System.out.println("\nBefore sorting (Products by price): ");
for (Product p : products) {
    System.out.println(p);
}

sortArray(products);

System.out.println("\nAfter sorting (Products by price): ");
for (Product p : products) {
    System.out.println(p);
} } }

class Product implements Comparable<Product> {
    private String name;
    private int price;

    public Product(String name, int price) {
        this.name = name;
        this.price = price;
    }
}
```

```
}  
  
@Override  
public int compareTo(Product other) {  
    return this.price - other.price;  
}  
  
@Override  
public String toString() {  
    return name + ": $" + price;  
} }  
}
```

OUTPUT:

```
Before sorting (Integers): [5, 3, 9, 1, 7]  
After sorting (Integers): [1, 3, 5, 7, 9]  
  
Before sorting (Strings): [John, Alice, Bob, David]  
After sorting (Strings): [Alice, Bob, David, John]  
  
Before sorting (Products by price):  
Laptop: $1000  
Phone: $800  
Tablet: $600  
Smartwatch: $200  
  
After sorting (Products by price):  
Smartwatch: $200  
Tablet: $600  
Phone: $800  
Laptop: $1000
```

CONCLUSION:

This program demonstrates the use of generics in Java to create a versatile sorting method for arrays of different types. By implementing the Comparable interface in the Product class, it enables sorting of custom objects based on specific criteria, such as price. The output shows the effective sorting of integers, strings, and products, highlighting the flexibility and reusability of the generic sorting method.

40

Write a program that counts the occurrences of words in a text and displays the words and their occurrences in alphabetical order of the words. Using Map and Set Classes.

PROGRAM CODE:

```
import java.util.*;

public class P40 {

    public static void main(String[] args) {

        Map<String, Integer> wordMap = new TreeMap<>();

        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter a text:");

        String text = scanner.nextLine();

        String[] words = text.toLowerCase().split("\\W+");

        for (String word : words) {

            if (!word.isEmpty()) {

                wordMap.put(word, wordMap.getOrDefault(word, 0) + 1);

            }

        }

        System.out.println("\nWord Occurrences (in alphabetical order):");

        Set<Map.Entry<String, Integer>> entrySet = wordMap.entrySet();

        for (Map.Entry<String, Integer> entry : entrySet) {

            System.out.println(entry.getKey() + ": " + entry.getValue());

        }

    }

}
```

OUTPUT:

```

Enter a text:
This is java Program.

Word Occurrences (in alphabetical order):
is: 1
java: 1
program: 1
this: 1

```

CONCLUSION:

This program demonstrates how to count and display the occurrences of words in a given text using Java's Map and Set classes. The words are stored in a TreeMap, ensuring that they are presented in alphabetical order. The use of `getOrDefault()` simplifies the counting process, showcasing efficient word frequency analysis.

41

Write a code which counts the number of the keywords in a Java source file. Store all the keywords in a HashSet and use the `contains()` method to test if a word is in the keyword set.

PROGRAM CODE:

```

import java.io.*;

import java.util.*;

public class P41 {

private static final HashSet<String> keywords = new HashSet<>();

static {

String[] keywordArray = {

"abstract", "assert", "boolean", "break", "byte", "case", "catch", "char", "class",

"const", "continue", "default", "do", "double", "else", "enum", "extends", "final",

"finally", "float", "for", "goto", "if", "implements", "import", "instanceof", "int",

"interface", "long", "native", "new", "package", "private", "protected", "public",

"return", "short", "static", "strictfp", "super", "switch", "synchronized", "this",

"throw", "throws", "transient", "try", "void", "volatile", "while"

```

```
};  
for (String keyword : keywordArray) {  
    keywords.add(keyword);  
} }  
  
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    System.out.print("Enter the path of the Java source file: ");  
    String filePath = scanner.nextLine();  
    try {  
        File file = new File(filePath);  
        Scanner fileScanner = new Scanner(file);  
        int keywordCount = 0;  
        while (fileScanner.hasNext()) {  
            String word = fileScanner.next();  
            if (keywords.contains(word)) {  
                keywordCount++;  
            }  
        }  
        System.out.println("Number of Java keywords in the file: " + keywordCount);  
        fileScanner.close();  
    } catch (FileNotFoundException e) {  
        System.out.println("File not found: " + filePath);  
    } } }
```

OUTPUT:

```
Enter the path of the Java source file: P40.java
Number of Java keywords in the file: 11
```

CONCLUSION:

This program demonstrates the use of a HashSet to efficiently count Java keywords in a source file. By reading each word from the file and checking for its presence in the set of keywords, it showcases how to utilize collections for rapid lookups. The result is the total number of keywords, providing a simple yet effective tool for analyzing Java code.

PART-V Exception Handling

- 24** Write a java program which takes two integers x & y as input, you have to compute x/y. If x and y are not integers or if y is zero, exception will occur and you have to report it.

PROGRAM CODE:

```
import java.util.Scanner;

public class Prac24 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
            System.out.print("Enter the value of x: ");
            int x = Integer.parseInt(scanner.nextLine());
            System.out.print("Enter the value of y: ");
            int y = Integer.parseInt(scanner.nextLine());
            int result = x / y;

            System.out.println("The result of " + x + " / " + y + " = " + result);
        }
        catch (NumberFormatException e) {
            System.out.println("Input is not a valid integer.");
        }
        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        }
        finally {
            scanner.close();
        }
    }
}
```

OUTPUT:

```
D:\java>javac Prac24.java

D:\java>java Prac24
Enter the value of x: 4
Enter the value of y: 0
Cannot divide by zero.
```

CONCLUSION:

To avoid the abnormal termination of the programme we can use the try catch block which helps the code to execute completely and also give us the exception which seems to be occurred.

- 25** Write a Java program that throws an exception and catch
• it using a try-catch block.

PROGRAM CODE:

```
public class Prac25 {
    public static void main(String[] args) {
        try {

            throw new Exception("This is a manually thrown exception.");
        }
        catch (Exception e) {

            System.out.println("Exception caught: " + e.getMessage());
        }
    }
}
```

```

    }
}
}

```

OUTPUT:

```

Exception caught: This is a manually thrown exception.
PS D:\SEM 3\JAVA\Practice\Exception>

```

CONCLUSION:

Here we throw a manual exception by customizing our own statement by which we can set according to our needs

- 26** Write a java program to generate user defined exception
- using “throw” and “throws” keyword.
- Also Write a java that differentiates checked and unchecked exceptions. (Mention at least two checked and two unchecked exceptions in program).

PROGRAM CODE:

```

class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class Prac26 {

```

```
public static void checkAge(int age) throws InvalidAgeException {  
    if (age < 18) {  
        throw new InvalidAgeException("Age is less than 18. Not eligible  
to vote.");  
    } else {  
        System.out.println("You are eligible to vote.");  
    }  
}  
  
public static void main(String[] args) {  
    try {  
  
        checkAge(16);  
    }  
    catch (InvalidAgeException e) {  
        System.out.println("Exception caught: " + e.getMessage());  
    }  
}
```

OUTPUT:

```
a24698ba\redhat.java\jdt_ws\Exception_efc4582e\bin' 'Prac26'  
Exception caught: Age is less than 18. Not eligible to vote.  
PS D:\SEM 3\JAVA\Practice\Exception>
```

CONCLUSION: Throw/Throws keywords are used to set user defined exceptions.

--	--

PART-VI File Handling & Streams

27

Write a program that will count the number of lines in each file that is specified on the command line. Assume that the files are text files. Note that multiple files can be specified, as in "java Line Counts file1.txt file2.txt file3.txt". Write each file name, along with the number of lines in that file, to standard output. If an error occurs while trying to read from one of the files, you should print an error message for that file, but you should still process all the remaining files.

PROGRAM CODE:

```
import java.io.*;

public class P27 {

    public static void main(String[] args) throws Exception {

        if (args.length == 0) {

            System.out.println("No file Found!");

        } else {

            for (int i = 0; i < args.length; i++) {

                try {

                    BufferedReader f = new BufferedReader(new FileReader(args[i]));

                    String j;

                    int count = 0;

                    while ((j = f.readLine()) != null) {

                        count++;

                    }

                    System.out.println("File name is : " + args[i] + " and Number of lines are : " + count);

                } catch (Exception e) {

                    System.out.println(e);

                }

            }

        }

    }

}
```

```
} } }
```

```
System.out.println("ID :23DCS013_JENIL CAPTAIN");
```

```
} }
```

OUTPUT:

```
File name is : file1.txt and Number of lines are : 5
File name is : file2.txt and Number of lines are : 4
File name is : file3.txt and Number of lines are : 9
```

CONCLUSION:

This Java program reads several files named by the command line arguments and counts the number of lines in each. If no files are provided as command-line arguments, it will print out the appropriate message. Exception handling ensures graceful error management during file reading, thus a stable program.

28

Write an example that counts the number of times a particular character, such as e, appears in a file. The character can be specified at the command line. You can use xanadu.txt as the input file.

PROGRAM CODE:

```
import java.io.BufferedReader;

import java.io.FileReader;

import java.io.IOException;

public class P28{

    public static void main(String[] args) {

        if (args.length < 2) {

            System.out.println("Usage: java P28 <character> <filename>");

            return; }

    }
```



```
char targetChar = args[0].charAt(0);

String fileName = args[1];

int count = 0;

try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {

    int ch;

    while ((ch = reader.read()) != -1) {

        if (ch == targetChar) {

            count++;

        }

    }

    System.out.println("The character '" + targetChar + "' appears " + count + " times in " +
        fileName);

} catch (IOException e) {

    System.out.println("Error reading " + fileName + ": " + e.getMessage());

}

System.out.println("ID :23DCS013_JENIL CAPTAIN");

}}
```

OUTPUT:

```
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> javac P28.java
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> java P28 a file1.txt
The character 'a' appears 16 times in file1.txt
```

CONCLUSION:

The Java program successfully counts the occurrences of a specified character in a given file, providing the result in a clear format. It handles file read errors gracefully, ensuring robust

performance even if issues arise during file access.

29

Write a Java Program to Search for a given word in a File. Also show use of Wrapper Class with an example.

PROGRAM CODE:

```
import java.io.BufferedReader;

import java.io.FileReader;

import java.io.IOException;

public class P29 {

    public static void main(String[] args) {

        if (args.length < 2) {

            System.out.println("Usage: java P29 <word> <filename>");

            return;

        }

        String searchWord = args[0];

        String fileName = args[1];

        Integer count = 0;

        try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {

            String line;

            while ((line = reader.readLine()) != null) {

                String[] words = line.split("\\W+");

                for (String word : words) {
```

```
if (word.equalsIgnoreCase(searchWord)) {  
    count++;  
} } }  
  
System.out.println("The word '" + searchWord + "' appears " + count + " times in " + fileName);  
} catch (IOException e) {  
  
    System.out.println("Error reading " + fileName + ": " + e.getMessage());  
}  
  
System.out.println("ID :23DCS013_JNEIL CAPTAIN");  
} }
```

OUTPUT:

```
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> javac P29.java  
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> java P29 and file3.txt  
The word 'and' appears 4 times in file3.txt
```

CONCLUSION:

This Java program effectively searches for a specified word in a given file and counts its occurrences. It demonstrates the use of the Integer wrapper class to manage the count, showcasing how wrapper classes can be used for object manipulation in Java.

30

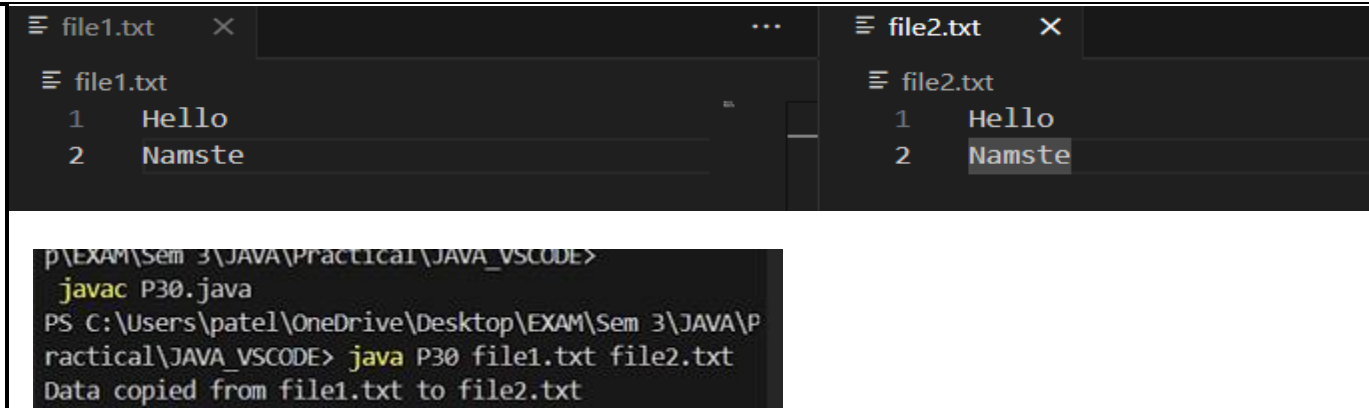
Write a program to copy data from one file to another file. If the destination file does not exist, it is created automatically.

PROGRAM CODE:

```
import java.io.FileReader;  
  
import java.io.FileWriter;  
  
import java.io.IOException;  
  
public class P30 {
```

```
public static void main(String[] args) {  
    if (args.length < 2) {  
        System.out.println("Usage: java P30 <source file> <destination file>");  
        return;  
    }  
  
    String sourceFile = args[0];  
    String destinationFile = args[1];  
  
    try (FileReader fr = new FileReader(sourceFile);  
        FileWriter fw = new FileWriter(destinationFile)) {  
        int ch;  
        while ((ch = fr.read()) != -1) {  
            fw.write(ch);  
        }  
  
        System.out.println("Data copied from " + sourceFile + " to " + destinationFile);  
    } catch (IOException e) {  
        System.out.println("Error: " + e.getMessage());  
    }  
  
    System.out.println("ID :23DCS013_JNEIL CAPTAIN");  
} }
```

OUTPUT:



```

p\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE>
javac P30.java
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> java P30 file1.txt file2.txt
Data copied from file1.txt to file2.txt

```

CONCLUSION:

This Java program efficiently copies data from a source file to a destination file, automatically creating the destination file if it does not already exist. It handles any potential I/O exceptions during the process, ensuring robust performance.

31

Write a program to show use of character and byte stream. Also show use of BufferedReader / BufferedWriter to read console input and write them into a file.

PROGRAM CODE:

```

import java.io.*;

public class P31 {

    public static void main(String[] args) {

        BufferedReader consoleReader = new BufferedReader(new InputStreamReader (System.in));

        String fileName = "output.txt";

        try (BufferedWriter fileWriter = new BufferedWriter(new FileWriter(fileName))) {

            System.out.println("Enter text (type 'exit' to finish):");

            String input;

            while (!(input = consoleReader.readLine()).equalsIgnoreCase("exit")) {

```

```
fileWriter.write(input);

fileWriter.newLine();

}

System.out.println("Data written to " + fileName);

} catch (IOException e) {

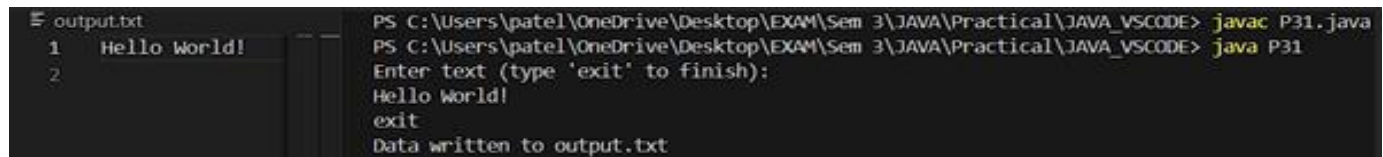
System.out.println("Error: " + e.getMessage());

}

System.out.println("ID :23DCS013_JENIL CAPTAIN");

} }
```

OUTPUT:



```
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> javac P31.java
PS C:\Users\patel\OneDrive\Desktop\EXAM\Sem 3\JAVA\Practical\JAVA_VSCODE> java P31
Enter text (type 'exit' to finish):
Hello World!
exit
Data written to output.txt
```

CONCLUSION:

This program effectively demonstrates the use of character streams via `BufferedReader` and `BufferedWriter` for reading console input and writing it to a file. It showcases how to handle text data efficiently while managing resources properly with `try-with-resources`.

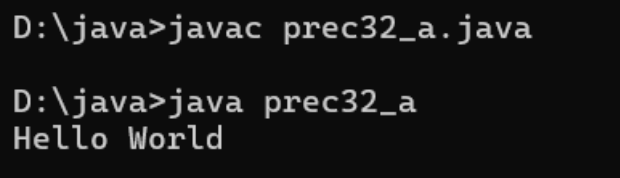
PART-VII Multithreading

- 32.** Write a program to create thread which display “Hello World” message. A. by extending Thread class B. by using Runnable interface.

PROGRAM CODE:

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Hello World");
    }
}

public class prec32_a {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();
    }
}
```

OUTPUT:

```
D:\java>javac prec32_a.java

D:\java>java prec32_a
Hello World
```

PROGRAM CODE:

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Hello World");
    }
}

public class prec32_b {
    public static void main(String[] args) {
        MyRunnable m1 = new MyRunnable();
        Thread t1 = new Thread(m1);
        t1.start();
    }
}
```

OUTPUT:

```
D:\java>javac prec32_b.java

D:\java>java prec32_b
Hello World
```

CONCLUSION:

In both examples, threads are used to execute the run() method. In the first case, MyThread directly extends Thread and overrides its run() method, allowing the thread to be started using t1.start(). In the second example, MyRunnable implements the Runnable interface, and its run() method is passed to a Thread object. Both approaches are valid for multithreading in Java, but implementing Runnable is preferred in most cases since it allows the class to extend other classes as well.

- 33.** Write a program which takes N and number of threads as an argument. Program should distribute the task of summation of N numbers amongst number of threads and final result to be displayed on the console.

PROGRAM CODE:

```
import java.util.Scanner;

class MyThread extends Thread {
    int start, end;
    static int sum = 0;

    // Constructor to initialize the range for each thread
    MyThread(int start, int end) {
        this.start = start;
```



```

        this.end = end;
    }

    // Synchronized method to safely add the sum from each thread
    static void addSum(int partialSum) {
        sum += partialSum;
    }

    // The task that each thread will perform
    public void run() {
        int partialSum = 0;
        for (int i = start; i <= end; i++) {
            partialSum += i;
        }
        addSum(partialSum); // Add the partial sum from this thread to the total sum
    }
}

public class prec33 {
    public static void main(String[] args) {
        int N, numThreads;
        Scanner s = new Scanner(System.in);

        // Getting input for the number N and number of threads
        System.out.print("Enter the number 'N': ");
        N = s.nextInt();
        System.out.print("Enter the number of threads to be used (should be less than or equal to N): ");
        numThreads = s.nextInt();

        // Array to hold the thread objects
        MyThread[] threads = new MyThread[numThreads];

        // Distribute the range of numbers across threads
        int range = N / numThreads;
        int start = 1, end;

        for (int i = 0; i < numThreads; i++) {
            end = (i == numThreads - 1) ? N : start + range - 1; // Ensure last thread handles the remainder
            threads[i] = new MyThread(start, end); // Create thread with the range of numbers
            threads[i].start(); // Start the thread
            start = end + 1;
        }

        try {
            for (int i = 0; i < numThreads; i++) {
                threads[i].join();
            }
        }
    }
}

```

```

    }
    } catch (InterruptedException e) {
        System.out.println("Thread interrupted.");
    }

    System.out.println("The sum of numbers from 1 to " + N + " is: " + MyThread.sum);
}
}

```

OUTPUT:

```

D:\java>javac prec33.java

D:\java>java prec33
Enter the number 'N': 3
Enter the number of threads to be used (should be less than or equal to N): 2
The sum of numbers from 1 to 3 is: 6

```

CONCLUSION:

In this program, multiple threads are used to calculate the sum of numbers from 1 to N. Each thread is responsible for a portion of the range, and the results are combined using a synchronized method to ensure thread safety. The use of Thread.join() ensures that the main program waits for all threads to complete before displaying the final sum. This approach speeds up the summation by distributing the task across multiple threads.

34. Write a java program that implements a multi-thread application that has three threads. First thread generates random integer every 1 second and if the value is even, second thread computes the square of the number and prints. If the value is odd, the third thread will print the value of cube of the number.

PROGRAM CODE:

```

class even extends Thread {
    int n;

    public even(int n) {
        this.n = n;
    }
}

```

```
        public void run() {
            System.out.println(n * n);
        }
    }

    class odd extends Thread {
        int n;

        public odd(int n) {
            this.n = n;
        }

        public void run() {
            System.out.println(n * n * n);
        }
    }

    public class prec34 {
        public static void main(String[] args) {
            even e;
            odd o;

            for (int i = 1; i <= 10; i++) {
                e = new even(i);
                o = new odd(i);
                try {
                    if (i % 2 == 0) {
                        e.start();
                    } else {
                        o.start();
                    }
                    Thread.sleep(1000);
                } catch (Exception ex) {
                    ex.printStackTrace();
                }
            }
        }
    }
```

OUTPUT:

```
D:\java>javac Prec34.java

D:\java>java Prec34
Odd cube of 1: 1
Even square of 2: 4
Odd cube of 3: 27
Even square of 4: 16
Odd cube of 5: 125
Even square of 6: 36
Odd cube of 7: 343
Even square of 8: 64
Odd cube of 9: 729
Even square of 10: 100
```

CONCLUSION:

In this program, separate threads calculate the square of even numbers and the cube of odd numbers from 1 to 10. The synchronized block ensures that the output is printed without interruption, and a delay of 1 second between thread executions is added for clarity. This demonstrates multithreading for handling different tasks based on even or odd numbers.

35. Write a program to increment the value of one variable by one and display it after one second using thread using sleep() method.

PROGRAM CODE:

```
class IncrementThread extends Thread {
    private int number;

    public IncrementThread(int number) {
        this.number = number;
    }

    @Override
```

```
public void run() {
    try {
        Thread.sleep(1000);
        number++;
        System.out.println("Incremented value after 1 second: " + number);
    } catch (InterruptedException e) {
        System.out.println("Thread was interrupted");
    }
}

public class IncrementValue {
    public static void main(String[] args) {
        int initialValue = 0;
        System.out.println("Initial value: " + initialValue);
        IncrementThread incrementThread = new IncrementThread(initialValue);
        incrementThread.start();
    }
}
```

OUTPUT:

```
PS D:\JAVA FILES\out> & 'C:\Program Files\Java\jdk-11.0.10\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages' -cp 'D:\JAVA FILES\out\IncrementValue.class' 'IncrementValue'
Initial value: 0
Incremented value after 1 second: 1
PS D:\JAVA FILES\out> █
```

CONCLUSION:

This program creates a thread that increments a variable's value by one after a one-second delay using the sleep() method. The initial value is printed first, followed by the updated value. It demonstrates how multithreading and delays can be handled in Java using the Thread class and sleep() function.

- 36.** Write a program to create three threads 'FIRST', 'SECOND', 'THIRD'. Set the priority of the 'FIRST' thread to 3, the 'SECOND' thread to 5(default) and the 'THIRD' thread to 7.

PROGRAM CODE:

```
class first extends Thread {  
    public void run() {  
        System.out.println("First");  
    }  
}
```

```
class Second extends Thread {  
    public void run() {
```

```
        System.out.println("Second");
    }
}

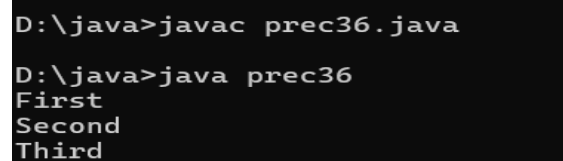
class Third extends Thread {
    public void run() {
        System.out.println("Third");
    }
}

public class prec36 {
    public static void main(String[] args) {
        first f = new first();

        Second s = new Second();
        Third t = new Third();

        f.setPriority(8);
        s.setPriority(4);
        t.setPriority(6);

        f.start();
        s.start();
        t.start();
    }
}
```

OUTPUT:

```
D:\java>javac prec36.java
D:\java>java prec36
First
Second
Third
```

CONCLUSION:

In this program, three threads (first, Second, and Third) are created and assigned different priorities. The thread priorities influence the likelihood of execution but do not guarantee the order in which they will run. This demonstrates how thread priorities can be set, though actual execution order depends on the system's thread scheduling.

37. Write a program to solve producer-consumer problem using thread synchronization.

PROGRAM CODE:

```
class Produce extends Thread {  
    int n;  
    boolean produced = false;  
  
    Produce(int n) {  
        this.n = n;  
    }  
  
    public synchronized void run() {  
        for (int i = 1; i <= n; i++) {
```



```
        while (produced) { // Wait if something is already produced
            try {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        System.out.println("Produced: " + i);
        produced = true;
        notify(); // Notify the consumer that production is done
    }
}

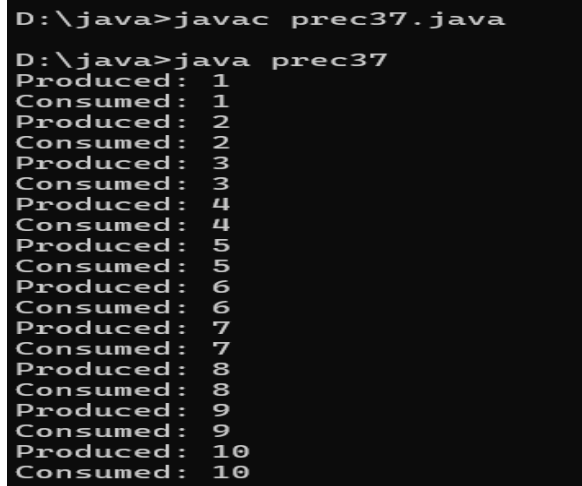
class Consume extends Thread {
    int n;
    Produce producer;

    Consume(int n, Produce producer) {
        this.n = n;
        this.producer = producer;
    }

    public synchronized void run() {
        for (int i = 1; i <= n; i++) {
            synchronized (producer) {
                while (!producer.produced) { // Wait until something is produced
                    try {
                        producer.wait();
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                    }
                }
                System.out.println("Consumed: " + i);
                producer.produced = false;
                producer.notify(); // Notify the producer to produce more
            }
        }
    }
}

public class prec37 {
    public static void main(String[] args) {
```

```
int n = 10;
Produce p = new Produce(n);
Consume c = new Consume(n, p);
p.start();
c.start();
}}
```

OUTPUT:

```
D:\java>javac prec37.java
D:\java>java prec37
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Produced: 6
Consumed: 6
Produced: 7
Consumed: 7
Produced: 8
Consumed: 8
Produced: 9
Consumed: 9
Produced: 10
Consumed: 10
```

CONCLUSION:

In this program, two threads—Produce and Consume—work together to produce and consume items sequentially. The producer thread creates items, and the consumer thread waits until an item is produced before consuming it. Synchronization and wait/notify methods are used to ensure proper coordination between the two threads, allowing them to work in an alternating fashion.

38

Design a Custom Stack using ArrayList class, which implements following functionalities of stack.

My Stack -list ArrayList<Object>: A list to store elements.

isEmpty(): boolean: Returns true if this stack is empty.

getSize(): int: Returns number of elements in this stack.

peek(): Object: Returns top element in this stack without removing it.

pop(): Object: Returns and Removes the top elements in this stack.

push(o: object): Adds new element to the top of this stack.

PROGRAM CODE:

```
import java.util.ArrayList;
```

```
class MyStack {
```

```
private ArrayList<Object> list = new ArrayList<>();
```

```
public boolean isEmpty() {
```

```
return list.isEmpty();
```

```
}
```

```
public int getSize() {
```

```
return list.size();
```

```
}
```

```
public Object peek() {
```

```
if (isEmpty()) {
```

```
return "Stack is empty";
```

```
}
```

```
return list.get(list.size() - 1);
```

```
}
```

```
public Object pop() {
```

```
if (isEmpty()) {  
    return "Stack is empty";  
}  
  
return list.remove(list.size() - 1);  
}  
  
public void push(Object o) {  
    list.add(o);  
} }  
  
public class P38 {  
    public static void main(String[] args) {  
        MyStack stack = new MyStack();  
        stack.push(10);  
        stack.push(20);  
        stack.push(30);  
        System.out.println("Top element is: " + stack.peek());  
        System.out.println("Popped element: " + stack.pop());  
        System.out.println("Popped element: " + stack.pop());  
        System.out.println("Is stack empty ? " + stack.isEmpty());  
        System.out.println("Current stack size: " + stack.getSize());  
        System.out.println("Top element now: " + stack.peek());  
    } }  
}
```

OUTPUT:

```
Top element is: 30
Popped element: 30
Popped element: 20
Is stack empty ? false
Current stack size: 1
Top element now: 10
```

CONCLUSION:

This program demonstrates the implementation of a custom stack using the ArrayList class in Java. It provides functionalities to push, pop, peek, check if the stack is empty, and get the current size of the stack. The program effectively showcases how to manage a dynamic collection of elements while adhering to stack principles.

39

Imagine you are developing an e-commerce application. The platform needs to sort lists of products based on different criteria, such as price, rating, or name. Each product object implements the Comparable interface to define the natural ordering. To ensure flexibility and reusability, you need a generic method that can sort any array of Comparable objects. Create a generic method in Java that sorts an array of Comparable objects. This method should be versatile enough to sort arrays of different types of objects (such as products, customers, or orders) as long as they implement the Comparable interface.

PROGRAM CODE:

```
import java.util.Arrays;

public class P39 {

    public static <T extends Comparable<T>> void sortArray(T[] array) {

        Arrays.sort(array);

    }

    public static void main(String[] args) {

        Integer[] numbers = {5, 3, 9, 1, 7};

        System.out.println("Before sorting (Integers): " + Arrays.toString(numbers));
```

```
sortArray(numbers);

System.out.println("After sorting (Integers): " + Arrays.toString(numbers));

String[] names = {"John", "Alice", "Bob", "David"};

System.out.println("\nBefore sorting (Strings): " + Arrays.toString(names));

sortArray(names);

System.out.println("After sorting (Strings): " + Arrays.toString(names));

Product[] products = {
    new Product("Laptop", 1000),
    new Product("Phone", 800),
    new Product("Tablet", 600),
    new Product("Smartwatch", 200)
};

System.out.println("\nBefore sorting (Products by price): ");

for (Product p : products) {

    System.out.println(p);

}

sortArray(products);

System.out.println("\nAfter sorting (Products by price): ");

for (Product p : products) {

    System.out.println(p);

} } }

class Product implements Comparable<Product> {
```

```
private String name;

private int price;

public Product(String name, int price) {
    this.name = name;
    this.price = price;
}

@Override
public int compareTo(Product other) {
    return this.price - other.price;
}

@Override
public String toString() {
    return name + ": $" + price;
} }
```

OUTPUT:

```
Before sorting (Integers): [5, 3, 9, 1, 7]
After sorting (Integers): [1, 3, 5, 7, 9]

Before sorting (Strings): [John, Alice, Bob, David]
After sorting (Strings): [Alice, Bob, David, John]

Before sorting (Products by price):
Laptop: $1000
Phone: $800
Tablet: $600
Smartwatch: $200

After sorting (Products by price):
Smartwatch: $200
Tablet: $600
Phone: $800
Laptop: $1000
```

CONCLUSION:

This program demonstrates the use of generics in Java to create a versatile sorting method for arrays of different types. By implementing the Comparable interface in the Product class, it enables sorting of custom objects based on specific criteria, such as price. The output shows the effective sorting of integers, strings, and products, highlighting the flexibility and reusability of the generic sorting method.

40

Write a program that counts the occurrences of words in a text and displays the words and their occurrences in alphabetical order of the words. Using Map and Set Classes.

PROGRAM CODE:

```
import java.util.*;

public class P40 {

    public static void main(String[] args) {

        Map<String, Integer> wordMap = new TreeMap<>();

        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter a text:");

        String text = scanner.nextLine();
```



```
String[] words = text.toLowerCase().split("\\W+");

for (String word : words) {
    if (!word.isEmpty()) {
        wordMap.put(word, wordMap.getOrDefault(word, 0) + 1);
    }
}

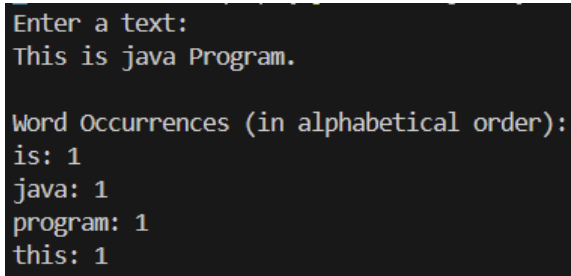
System.out.println("\nWord Occurrences (in alphabetical order):");

Set<Map.Entry<String, Integer>> entrySet = wordMap.entrySet();

for (Map.Entry<String, Integer> entry : entrySet) {

    System.out.println(entry.getKey() + ": " + entry.getValue());

} } }
```

OUTPUT:A screenshot of a terminal window showing the output of the Java program. The input text is "This is java Program.". The output shows the word occurrences in alphabetical order: "is: 1", "java: 1", "program: 1", and "this: 1".

```
Enter a text:
This is java Program.

Word Occurrences (in alphabetical order):
is: 1
java: 1
program: 1
this: 1
```

CONCLUSION:

This program demonstrates how to count and display the occurrences of words in a given text using Java's Map and Set classes. The words are stored in a TreeMap, ensuring that they are presented in alphabetical order. The use of getOrDefault() simplifies the counting process, showcasing efficient word frequency analysis.

41

Write a code which counts the number of the keywords in a Java source file. Store all the keywords in a HashSet and use the contains () method to test if a word is in the keyword set.

PROGRAM CODE:

```
import java.io.*;
```

```
import java.util.*;

public class P41 {

private static final HashSet<String> keywords = new HashSet<>();

static {

String[] keywordArray = {

"abstract", "assert", "boolean", "break", "byte", "case", "catch", "char", "class",

"const", "continue", "default", "do", "double", "else", "enum", "extends", "final",

"finally", "float", "for", "goto", "if", "implements", "import", "instanceof", "int",

"interface", "long", "native", "new", "package", "private", "protected", "public",

"return", "short", "static", "strictfp", "super", "switch", "synchronized", "this",

"throw", "throws", "transient", "try", "void", "volatile", "while"

};

for (String keyword : keywordArray) {

keywords.add(keyword);

} }

public static void main(String[] args) {

Scanner scanner = new Scanner(System.in);

System.out.print("Enter the path of the Java source file: ");

String filePath = scanner.nextLine();

try {

File file = new File(filePath);

Scanner fileScanner = new Scanner(file);
```

```
int keywordCount = 0;

while (fileScanner.hasNext()) {

String word = fileScanner.next();

if (keywords.contains(word)) {

keywordCount++;

} }

System.out.println("Number of Java keywords in the file: " + keywordCount);

fileScanner.close();

} catch (FileNotFoundException e) {

System.out.println("File not found: " + filePath);

} } }
```

OUTPUT:

```
Enter the path of the Java source file: P40.java
Number of Java keywords in the file: 11
```

CONCLUSION:

This program demonstrates the use of a HashSet to efficiently count Java keywords in a source file. By reading each word from the file and checking for its presence in the set of keywords, it showcases how to utilize collections for rapid lookups. The result is the total number of keywords, providing a simple yet effective tool for analyzing Java code.

